

Student Recruitment Platform

System Planning Document

1. Project Overview

The Student Recruitment Platform is a centralized digital ecosystem designed to connect **Students, Agents, and Universities** efficiently.

The system focuses on:

- Structured application workflows
- Centralized data management
- Admin-controlled operations
- Scalable and secure architecture

Goal:

To streamline the admission process, empower agents, and maintain full administrative control.

2. System Objectives

- Automate the student admission workflow
 - Enable agents to manage applications efficiently
 - Provide structured course and application data
 - Ensure secure and scalable architecture
 - Reduce manual errors through automation
-

3. Technology Stack

Frontend

- React.js (TypeScript)
- Next.js
- Tailwind CSS / Material UI

Backend

- Node.js + Express.js (TypeScript)

Database

- PostgreSQL

Cloud & DevOps

- AWS / VPS
 - Docker (optional)
 - Nginx
-

4. System Architecture

Architecture Type

- Client-Server Architecture
- Modular & Scalable

Layers

1. Frontend Layer
 2. Backend Layer
 3. Database Layer
 4. Integration Layer
 5. Storage Layer
-

5. User Roles & Access Control

Roles

- Student
- Agent
- Admin
- University

Security

- Role-Based Access Control (RBAC)
 - JWT Authentication
 - Secure APIs
-

6. Core Modules

6.1 Student Module

- Course & career input
- Guided application flow
- Dashboard (tracking, uploads, counselling)

6.2 Agent Module

- Application management
- Document tracking
- Commission tracking
- Deadline alerts
- Branding support

6.3 University Module

- Application viewing (restricted)
- Analytics dashboard
- Ranking (optional)

6.4 Admin Panel

- Full system control
 - User & application management
 - Offer letters
 - Commission & subscriptions
 - Deadlines & rankings
-

7. Application Management System

Lifecycle

Draft → Submitted → Review → Approved → Offer → Deposit → Visa → Enrolled

Features

- Multi-step forms
 - Status tracking
 - Document validation
 - Error handling
-

8. Payment System

- Agent subscriptions
 - Payment gateway integration
 - Payment tracking
-

9. Communication System

- WhatsApp integration
 - Email notifications
 - Bulk messaging
 - Automated alerts
-

10. Database Design

Entities

- Users
- Profiles (Student/Agent)
- Universities
- Courses
- Applications
- Documents
- Payments
- Notifications
- Logs

Relationships

- User → Profile
 - Application → Student + Agent + Course
 - Course → University
-

11. Backend Architecture

Structure:

- Controllers
- Services
- Repositories

- Middleware

Flow:

Route → Controller → Service → Database

Security:

- JWT
 - bcrypt hashing
 - Validation
-

12. Frontend Architecture

- Component-based structure
 - Role-based dashboards
 - API services
 - Reusable UI
-

13. Security & Compliance

- Encrypted passwords
 - Secure APIs
 - File validation
 - Activity logging
-

14. Scalability Strategy

- Modular architecture
 - Database indexing
 - Load balancing
 - CDN usage
 - Background jobs
-

15. Development Phases

1. Planning
2. Core Development
3. Advanced Features

4. Testing
 5. Deployment
-

16. Domain & Deployment

Domains:

- educratsglobal.com
- unibridgex.com

Hosting:

- Frontend: Vercel / Netlify
 - Backend: AWS EC2 / VPS
 - Database: AWS RDS
-

17. References

- Node.js Docs
 - PostgreSQL Docs
 - AWS Guides
 - WhatsApp Business API
-

18. Conclusion

This platform provides a **structured, secure, and scalable solution** for managing student admissions while improving efficiency across all stakeholders.

Student Recruitment Platform: System Planning Document1. Executive Summary and Project Goal

This document outlines the plan for a **Student Recruitment Platform**, a centralized digital ecosystem connecting **Students, Agents, and Universities** to streamline the admission process. The platform is designed for **structured application workflows, centralized data management, and admin-controlled, scalable, and secure operations**.

Core Goal: To create a secure, scalable solution that automates and streamlines the admission process, empowers agents, and maintains complete administrative oversight.2.

Key System Objectives

- Automate and standardize the student admission workflow.

- Provide agents with tools for efficient application management (e.g., commission tracking, deadline alerts).
- Ensure secure and scalable architecture with robust data management.
- Reduce manual intervention and errors through process automation.
- Offer structured access to course and application data for all stakeholders.

3. Technology and Architecture3.1 Technology Stack

Category	Technologies
Frontend	React.js (TypeScript), Next.js, Tailwind CSS / Material UI
Backend	Node.js + Express.js (TypeScript)
Database	PostgreSQL
Cloud/DevOps	AWS / VPS, Docker (optional), Nginx

3.2 System Architecture

- **Type:** Client-Server Architecture (Modular & Scalable).
- **Layers:** Frontend, Backend, Database, Integration, and Storage.
- **Backend Flow:** Route → Controller → Service → Repository/Database.

4. User Roles and Security4.1 User Roles

- Student
- Agent
- University
- Admin

4.2 Security Measures

- **Authentication:** JWT Authentication.
- **Authorization:** Role-Based Access Control (RBAC).
- **Data Security:** Secure APIs, encrypted passwords (bcrypt hashing), file validation, and activity logging.

5. Core System Modules and Features

Module	Key Functions
Student	Guided application flow, course/career input, dashboard (tracking, uploads, counselling).
Agent	Application management, document/commission tracking, deadline alerts, branding support.

University	Restricted application viewing, analytics dashboard, optional ranking management.
Admin Panel	Full system control, user/application/offer letter management, commission/subscription control, deadlines/rankings.

5.1 Application Management

- **Lifecycle:** Draft → Submitted → Review → Approved → Offer → Deposit → Visa → Enrolled.
- **Features:** Multi-step forms, status tracking, document validation, and robust error handling.

5.2 Supporting Systems

- **Payment:** Agent subscriptions, integrated payment gateway, payment tracking.
- **Communication:** Email notifications, WhatsApp integration, automated alerts, bulk messaging.

6. Data Management and Scalability

6.1 Database Design (Entities)

Users, Profiles (Student/Agent), Universities, Courses, Applications, Documents, Payments, Notifications, and Logs.

- **Key Relationships:** User → Profile; Application → Student + Agent + Course; Course → University.

6.2 Scalability Strategy

- Modular architecture for decoupled development.
- Database indexing for performance.
- Load balancing and CDN usage for handling traffic.
- Implementation of background jobs for heavy processing.

7. Development and Deployment

7.1 Development Phases

1. Planning
2. Core Development
3. Advanced Features
4. Testing
5. Deployment

7.2 Deployment Environment

- **Frontend Hosting:** Vercel / Netlify.
- **Backend Hosting:** AWS EC2 / VPS.
- **Database:** AWS RDS (or equivalent).
- **Potential Domains:** educratsglobal.com, unibridgex.com.

